Rig Robot: one quadruped, two jobs: rust repair and gas-leak response

Unitree Go2 + 0–0.45 m lift mast + UR3 arm on an offshore oil rig in Isaac Sim 6.2. One state machine, one camera, one arm, one decision log. Every step is decided by what it sees and senses.

Agentverse 2026 · Robotics track
Technical summary

USER AND PROBLEM

User: asset-integrity and maintenance crews on offshore platforms. Two chronic jobs send people into the hazard zone. **Corrosion** on topside pipework is found from rope access or scaffolding and fixed by hand. **Small gas leaks** at bolted flanges are found by a person carrying a gas detector, often right next to the leak.

The work is **dangerous** (height, weather, flammable gas), **costly** (people flown offshore for routine work) and **never finished** (kilometres of pipe and hundreds of flanges per platform). A robot that patrols, judges and does the routine fix, and hands only the doubtful or dangerous cases to a person, keeps people out of the hazard zone.

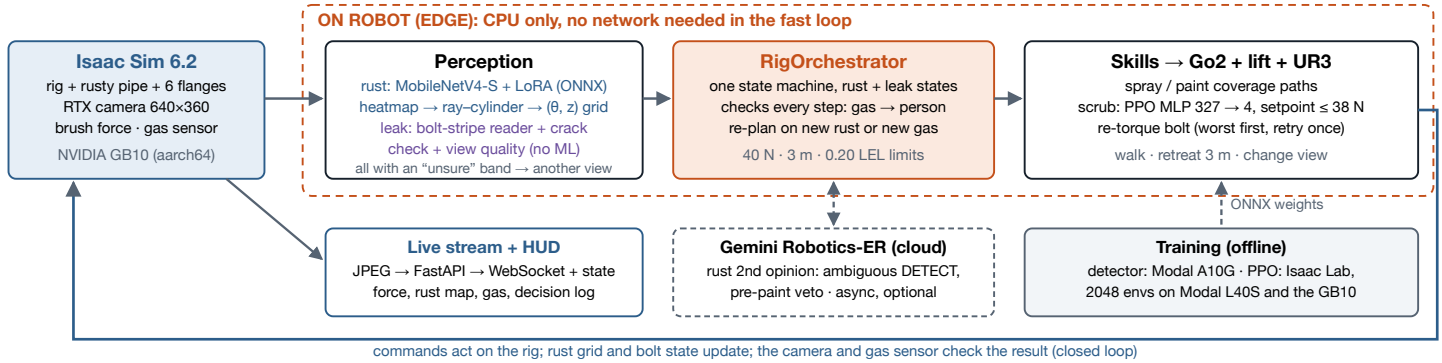
TWO JOBS, ONE PLATFORM

Rust repair. The detector heatmap is projected onto a (θ, z) grid on the pipe. The robot sprays, soaks 30 s, scrubs with a PPO policy (wear ~ force × speed, 4× faster once soaked), verifies with the detector and Gemini, then paints.

Gas-leak response. A gas check runs at every patrol stop and before every step. If gas is above 0.05 LEL, the robot walks to the nearest flange. A torque-stripe reader measures how far each bolt has turned and a classical crack check looks at the flange face. A loose bolt gets re-torqued and verified (stripe aligned, gas falling). A crack is never touched: the robot backs off and escalates.

Shared: gas pre-empts everything, even mid-scrub: the robot pauses the scrub, fixes the leak and resumes. Every transition, with its evidence, goes to `decisions.jsonl` and the live HUD.

SYSTEM ARCHITECTURE



RUST `PATROL` → `GAS_CHECK` → `DETECT` ⇄ SECOND_VIEW → `APPROACH` → `SPRAY` → `SOAK 30 s` (new rust ⇒ re-spray) → `SCRUB (PPO)` ⇄ BACK_OFF → `VERIFY` (≤ 3 cycles) → `PAINT` → `VERIFY_PAINT` → `DONE`

LEAK `GAS_CHECK` (> 0.05 LEL) → `LEAK_APPROACH` → `LEAK_INSPECT` ⇄ LEAK_SECOND_VIEW → `RETORQUE` ⇄ `LEAK_VERIFY` → fixed: resume paused job · crack / unsure ⇒ ESCALATE_TO_HUMAN · ≥ 0.20 LEL or sensor fault ⇒ `GAS_RETREAT`

WHERE EACH MODEL RUNS

Component	Runs on	Latency / rate
Rust classifier (2.49 M, 0.19 GMACs)	robot CPU, ONNX	1.1 ms (4 thr) · 2.6 ms (1 thr)
Rust localizer (patches on 640×360)	robot CPU	batched · every 5 s in soak
Leak check (8 bolt stripes + crack)	robot CPU, numpy	~90 ms / frame* · 3 frames/view
Scrub policy (MLP, 327 → 4)	robot CPU, ONNX	0.012 ms* · 10 Hz loop
RigOrchestrator + skills	robot CPU, Python	gas + person check every step
Edge TensorRT on GB10	GB10 GPU	TBD
Gemini Robotics-ER 2 preview	cloud API, async	TBD · 20 s timeout
Isaac Sim 6.2 render + stream	GB10, 121 GB unified	RTX · JPEG over WebSocket
Training: detector / PPO	Modal A10G / L40S, GB10	PPO ~45k / ~15k env-steps/s

*Apple M5 Max CPU, batch 1. Detector latency is from the bake-off. GB10: 150 PPO iterations × 2048 envs in 524 s. Modal cannot render (no Vulkan), so all RTX rendering runs on the GB10.

KEY RESULTS

RUST DETECTOR · MOBILENETV4-S + LORA R=16 (0.26 M TRAINABLE)	
Real-photo test (881): F1 / recall / AUROC	**0.977** / 0.983 / 0.998
Held-out Isaac render crops (v1 → v2 model)	10/13 → **13/13**

SCRUB PPO · RIG GEOMETRY · NEW PATCH EACH EPISODE	V1	V2
Success (< 5 % rust left) · episodes	100 % · 513	97.6 % · 1026
Episodes with force > 40 N	76.8 %	**1.0 %**
Mean / p95 episode-max force	49.9 / 70.1 N	**29.9 / 35.7 N**

GAS-LEAK JOB (CPU MOCK RENDERS AT THE ISAAC CAMERA GEOMETRY)	
Crack check: cracks found · false alarms on clean	**40/40** · **0/40** (6/40 clean → unsure)
Bolt-stripe angle error · confidence (clean lens)	≤ 0.019 rad (~1°) · ≥ 0.95
Loose bolts 1 and 5 → re-torque both, gas clears	DONE
Bolt 2 loosens mid-scrub → pause, re-torque, resume	DONE
Cracked flange · dirty lens (2 extra views)	ESCALATE · ESCALATE

The v2 scrub gives up 2.4 points of success to stay inside the force budget. The v2 demo render (Isaac Lab RTX on the GB10) peaks at 31.9 N and leaves 2.5 % residual, which it flags for touch-up. The full two-job loop in Isaac 6.2 with the trained models is TBD.

CRITICAL FAILURE MODES → RESPONSE (ON THE HUD)

Paint over rust	VERIFY needs < 5 % rust, ≥ 80 % of the target visible and ≤ 5 % uncertain cells. Gemini can veto. Still rusty after 3 cycles → **ESCALATE** with the residual cells.
Tighten a cracked flange	Crack p ≥ 0.5 → never a robot repair; back off 3 m, critical alert. While a crack is possible (unsure), no bolt is touched.
Gas at the limit	≥ 0.20 LEL, or a failed sensor → **GAS_RETREAT** (fail-safe), then escalate. No tightening starts at ≥ 0.20 LEL.
Leak mid-rust-job	> 0.05 LEL → pause the scrub, fix the leak, resume the same state.
Over-force	> 40 N → **BACK_OFF**: retract 5 cm, press × 0.7; 3× → escalate. 38 N setpoint clamp outside the network.
Can't see well	Unsure rust or crack → up to 2 extra views, then a human. A dirty lens means "no crack" is not trusted. No frame → **HOLD**.
Person nearby	Worker < 3 m → **PAUSE_PERSON**; resume when clear.

SAFETY DESIGN AND LIMITS

- **Order of checks before every step:** gas sensor working → gas below the limit → gas clear during rust work → person clear. The most severe hazard wins.
- **Hard limits live outside the learned policy** (40 N gate, 38 N clamp, 3 m, 0.20 LEL, retry caps). In doubt, the robot looks again, holds or asks a person. The cloud is never in the fast loop.
- **Limits:** simulation only. The robot is posed kinematically (not PhysX), the brush force is a spring model, and bolt torque and thread friction are not simulated. The gas model and thresholds are illustrative, not a site standard. The leak checks have not yet been re-tested on Isaac stream frames. The sim-render rust test is small (13 crops).

IN PROGRESS TBD

- **Unit 2 "gas" robot:** point methane (%LEL) and H_2S sensors, an OGI-style camera, an acoustic leak sensor, cast-and-surge source localisation.
- **3D gas plume dispersion** with leak rate (orifice equation) and volumetric flow in a corner camera window. Also a **multi-robot fleet planner** with gas pre-emption, **storm adaptation** and **Go2 walking**.

Sources: `detection/RESULTS.md`, `policy/SCRUB.md`, `runs/scrub_v2/` and `runs/scrub_watson/eval_rig_meta_forcepatch.json`, `runs/scrub_demo/`, `docs/leak-diagrams.md`, `python -m detection.eval_leak_sim`, `python -m policy.leak.mock`, `integration/SIM_BRIDGE.md`. Live stream: rig.zeecloud.org. TBD = still being produced.